

KAPSAMLI ÖĞRENİM REHBERİ

To-Do API

Backend — Sıfırdan

Kavramsal Temel + Faz 1-3 Uygulama
+ *Derinlemesine JWT & Authentication*

İçindekiler

KISIM I — Kavramsal Temel

Backend, client-server, HTTP, method'lar, status code'lar
REST, JSON, API, veritabanı, auth kavramları
Teknoloji yığını: ne, neden, alternatifi

KISIM II — Uygulama: Faz 1-3

Faz 1: İskelet sunucu (TS, Express, tsx, git)
Faz 2: Veritabanı (PostgreSQL, pg, SQL, pool)
Faz 3: Authentication (Zod, bcrypt, JWT, middleware)

KISIM III — JWT & Auth Derinlemesine

Neden token? Stateless problemi
Token'ın anatomisi, imza, doğrulama
requireAuth satır satır + tam akış
Sık sorulan sorular ve tuzaklar

KISIM IV — Referanslar

Dosya haritası, istek döngüsü, sözlük, commit'ler

KISIM V — Veri Nereye Gidiyor?

Dönen değerler zinciri: ne alır, ne döndürür
Token'ı açmak: header, payload, signature

KISIM I

Kavramsal Temel

Kod yazmadan önce: backend nedir, internet nasıl konuşur, ve neden bu teknolojileri seçtik.

BÖLÜM 1

Backend ve internetin temelleri

Kod yazmaya başlamadan önce birkaç temel mekaniği oturtmamız gerekiyor. Bu bölüm, projenin geri kalanında üzerine inşa edeceğin sözcük dağarcığını veriyor.

Backend nedir

Bir uygulamanın kullanıcının göremediği tarafı. Arayüz (frontend) tarayıcıda ya da telefonda çalışır; backend uzak bir sunucuda durur ve üç şey yapar: veriyi saklar (veritabanı), iş kurallarını uygular (bir kullanıcı başkasının to-do'sunu silebilir mi?), ve dış dünyaya bir konuşma kanalı sunar (API). Frontend güzel görünmekle, backend doğru olmakla yükümlüdür — backend kaynak gerçek (source of truth) olmak zorundadır.

Client-Server modeli

Tüm web bu basit model üzerine kurulu: bir taraf istek (request) yapar, diğer taraf cevap (response) verir. İstek yapan client (tarayıcı, mobil uygulama, başka bir sunucu), cevap veren server. Aralarındaki tek söz ortak protokol: HTTP. Sen backend'i Node ile yazarken client'i biri Swift ile yazabilir; ikisi de aynı dilden konuştuğu sürece anlaşılır. Bizim API'mizin client-agnostic olması tam bu demek.

HTTP — internetin dili

Üç özelliği aklında olsun:

Metin tabanlı İstek ve cevaplar düz metindir, insan okuyabilir.

Request-Response Her iletişim tek istek + tek cevap. Client ister, server cevaplar, kapanır.

Stateless Server önceki isteği HATIRLAMAZ. Her istek kendi başına anlamlı olmalı. Bu yüzden her istekte kim olduğunu kanıtlaman gerekir — JWT'nin var oluş sebebi.

HTTP method'ları (fiiller)

GET Kaynağı oku. Değiştirmez. Örn: GET /todos

POST Yeni kaynak yarat. Örn: POST /todos

PUT Kaynağın tamamını yerine koy. Idempotent.

PATCH Kaynağın bir kısmını güncelle.

DELETE Kaynağı sil. Idempotent.

Idempotency

Bir method'u aynı parametrelerle bir kez ya da on kez çağırdığında sonuç aynıysa, o method idempotent'tir. GET, PUT, DELETE idempotent; POST değildir (tekrar çağırırsa iki kayıt yaratır). Network'te istek tekrar gönderilebildiği için önemli.

HTTP status code'ları

Aile	Anlam	Örnekler
2xx	Başarı	200 OK, 201 Created, 204 No Content
3xx	Yönlendirme	301 Moved, 304 Not Modified
4xx	Client hatası	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict
5xx	Server hatası	500 Internal Server Error, 503 Unavailable

401 vs 403 — sık karışır

401: 'Sen kimsin? Kanıtla.' Token yok ya da geçersiz. 403: 'Kim olduğunu biliyorum ama bu işi yapma yetkin yok.'

BÖLÜM 2

REST, JSON, API ve veritabanı

REST nedir

REST, bir API'yi tasarlarırken uyulan bir üslup (kurallar bütünü) — teknoloji değil. Temel fikri: API'yi kaynaklar etrafında tasarla, bu kaynaklar üzerinde HTTP'nin standart fiilleriyle iş yap. Bizim kaynaklarımız user ve todo:

GET	/todos	→ tüm to-do'ları listele
POST	/todos	→ yeni to-do yarat
GET	/todos/42	→ 42 ID'li to-do'yu getir
PATCH	/todos/42	→ 42'yi güncelle
DELETE	/todos/42	→ 42'yi sil

URL'de fiil olmaz ('getTodos' REST değil). Fiil zaten HTTP method'unun kendisidir. Kaynak adları çoğul ve küçük harf olur.

JSON nedir

API'lerin veri alışverişinde kullandığı metin formatı. Dilden bağımsız standart:

```
{
  "id": 42,
  "title": "PostgreSQL JOIN'leri öğren",
  "completed": false,
  "createdAt": "2026-06-20T10:00:00Z"
}
```

Tipler: string, number, boolean, null, object, array. Anahtarlar çift tırnak içinde, comment yok.

API vs REST API

API çok geniş bir terim: bir yazılımın başka bir yazılıma sunduğu her türlü arayüz. REST API ise: HTTP üzerinden, REST üslubuyla tasarlanmış, genelde JSON konuşan bir web API'si. Bu projede inşa ettiğin tam olarak bu.

Veritabanı türleri

Tür	Örnek	Ne için iyi
İlişkisel (SQL)	PostgreSQL, MySQL	Yapısı belli, ilişkili, tutarlılık kritik veri
Document	MongoDB	Şeması değişken, içiçe nested veri
Key-value	Redis	Cache, oturum, hızlı basit erişim

Graph	Neo4j	Sosyal ağ, öneri sistemleri
-------	-------	-----------------------------

Bizim verimiz yapısı belli (email, şifre, başlık...), açık ilişkili (her to-do bir user'a ait) ve tutarlılık kritik (aynı email iki kez olamaz). İlişkisel DB bu üçünü garantilerle (şema, foreign key, transaction) karşılar. O yüzden PostgreSQL.

Authentication vs Authorization

Authentication

Sen kimsin? Email+şifre ile kimlik kanıtlama (login). Çıktısı: 'bu kullanıcı 42 numara.'

Authorization

Sen ne yapabilirsin? Kimliği bilinen kullanıcının bir kaynağa erişme hakkı var mı? '42, 7 ID'li to-do'yu silebilir mi? Hayır, başkasının.'

Önce authN (sen kimsin — login + token), sonra authZ (yapmaya çalıştığın işe hakkın var mı). Bu ayrım hem mülakatta hem üretimde kritik.

BÖLÜM 3

Teknoloji yığını

Her tercih bilinçli. Ne, neden, alternatifi:

Node.js

JavaScript'i sunucuda çalıştıran runtime (V8 motoru). Tek thread ama event loop + non-blocking I/O sayesinde binlerce eşzamanlı isteği kaldırır. Alternatifler: Python (FastAPI), Go, Java (Spring), .NET.

TypeScript

JavaScript + tip sistemi. Hataları yazarken yakalar; tipler runtime'da silinir (type erasure). Alternatif: düz JS — ama TS profesyonel standart.

Express

Node üzerinde minimalist web framework'ü. 'Magic az', şeffaf — öğrenmek için ideal. Alternatifler: Fastify, Koa, NestJS.

PostgreSQL

Güçlü, açık kaynak ilişkisel DB. ACID garantileri, zengin tip sistemi. Alternatifler: MySQL, SQLite, SQL Server.

Neon

PostgreSQL'i bulutta veren serverless servis. Kendi makinene kurmana gerek yok. Alternatifler: Supabase, Railway, Render.

pg

Node ↔ PostgreSQL driver'ı. Ham SQL yazarsın (ORM yok — SQL'i öğrenmek için bilinçli tercih). Alternatifler (ORM): Prisma, TypeORM, Drizzle.

Zod

TypeScript-öncelikli şema doğrulama. Tek şemadan hem runtime doğrulama hem TS tipi. Alternatifler: Joi, Yup, Valibot.

bcryptjs

Şifreleri tek yönlü hash'ler. Salt + kasıtlı yavaşlık. Alternatifler: argon2 (daha modern), scrypt.

jsonwebtoken

Login sonrası imzalı kimlik token'ı (JWT) üretir/doğrular. Alternatif: server-side session (Redis/DB'de oturum).

dotenv

Sırları .env'den process.env'e yükler. Alternatif: Node 20.6+ yerleşik --env-file.

KISIM II

Uygulama: Faz 1-3

Kavramları gerçek koda dökmek. Her dosya, ne işe yaradığı, nasıl çalıştığı.

FAZ 1

İskelet Sunucu

Hedef: npm run dev ile localhost'ta ayağa kalkan, TypeScript + Express sunucusu. Düzgün TS projesi, anlamlı klasör yapısı, hızlı geliştirme döngüsü, git'e bağlı repo.

package.json ve bağımlılıklar

dependency vs devDependency

dependencies → uygulama çalışırken gerekli (express, pg, zod, bcryptjs, jsonwebtoken, dotenv). devDependencies → yalnız geliştirme/build zamanında (typescript, tsx, @types/*). npm install --production sadece dependencies kurar.

tsconfig.json

backend/tsconfig.json

TypeScript derleyici yapılandırması

```
{
  "compilerOptions": {
    "target": "ES2022", "module": "commonjs",
    "rootDir": "./src", "outDir": "./dist",
    "strict": true, "esModuleInterop": true,
    "skipLibCheck": true, "resolveJsonModule": true,
    "forceConsistentCasingInFileNames": true, "sourceMap": true
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules", "dist"]
}
```

tsc vs tsx

	tsc	tsx
Ne yapar?	TS → JS derler	TS'i doğrudan çalıştırır
Çıktı	dist'e JS	Disk çıktısı yok
Sunucu başlatır?	Hayır	Evet (watch ile restart)
Ne zaman?	Production build	Geliştirme

Kural: dev'de tsx, prod'da tsc + node.

İlk sunucu ve script'ler

backend/src/server.ts

Giriş noktası (Faz 1 hali)

```
import express, { Request, Response } from "express";
const app = express();
const PORT = 3000;

app.get("/health", (req: Request, res: Response) => {
  res.json({ status: "ok", uptime: process.uptime() });
});

app.listen(PORT, () => console.log(`Server on http://localhost:${PORT}`));
```

```
"scripts": {
  "dev": "tsx watch src/server.ts",
  "build": "tsc",
  "start": "node dist/server.js"
}
```

Conventional Commits

Format: type(scope): summary. feat, fix, chore, refactor, docs, test. İngilizce, emir kipi, küçük harf. Örn: feat(backend): scaffold express + typescript server

FAZ 2

Veritabanı

Hedef: kalıcı bellek. Kullanıcılar ve to-do'lar Neon (cloud PostgreSQL) üzerinde yaşıyor; sunucu restart etse de veri kalır.

İlişkisel model ve veri tipleri

Veriyi tablolarda saklar: tablo=relation, satır=kayıt, sütun=alan. Bir sütundaki tüm hücreler aynı tipte. Başlıca tipler:

Tip	Saklar
SERIAL	Otomatik artan tam sayı (DB verir)
INTEGER	Tam sayı
TEXT	Sınırsız metin
BOOLEAN	true / false
TIMESTAMPTZ	Saat dilimli zaman damgası (üretimde her zaman bu)

PK, FK ve constraint'ler

PRIMARY KEY Satırın benzersiz kimliği. Otomatik NOT NULL+UNIQUE+index. SERIAL ideal.

FOREIGN KEY Bir sütun başka tablonun PK'sını işaret eder. Hayalet referans imkansız.

NOT NULL Sütun her satırda dolu olmalı.

UNIQUE Değer tabloda tek. Otomatik index kazandırır.

DEFAULT INSERT'te verilmezse koyulacak değer (now(), false).

ON DELETE CASCADE

Parent silinince child satırlar otomatik silinir. Bizde: user silinince to-do'ları da gider. Alternatifler: SET NULL, RESTRICT.

Şema

backend/src/db/schema.sql

Tabloların tanımı — yapının kaynak gerçeği

```
CREATE TABLE users (  
  id          SERIAL PRIMARY KEY,  
  email       TEXT NOT NULL UNIQUE,  
  password    TEXT NOT NULL,      -- hash saklanır, düz şifre asla  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE todos (  
  id          SERIAL PRIMARY KEY,  
  user_id     INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  title       TEXT NOT NULL,  
  completed   BOOLEAN NOT NULL DEFAULT false,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE INDEX idx_todos_user_id ON todos(user_id);
```

`user_id` neden INTEGER? Var olan `users.id`'yi işaret ediyor; FK tipi PK tipiyle aynı olmalı. Index neden `user_id`'ye? En sık sorgu 'şu kullanıcının to-do'ları' (`WHERE user_id=$1`); index'siz tüm tablo taranır. PK/UNIQUE otomatik index'li, FK değil.

.env, pg ve connection pool

backend/src/db/pool.ts

PostgreSQL bağlantı havuzu

```
import { Pool } from "pg";  
import dotenv from "dotenv";  
dotenv.config();  
  
export const pool = new Pool({  
  connectionString: process.env.DATABASE_URL,  
});
```

Connection pool

Her sorguda yeni bağlantı açmak pahalı (TCP+SSL+auth). Pool önceden açılmış bağlantıları hazır tutar; sorgu gelince verir, bitince havuza iade eder. Tek pool instance tüm uygulamada paylaşılır. Web için Client değil, Pool.

Temel SQL

```
INSERT INTO users (email, password) VALUES ($1, $2)
RETURNING id, email, created_at;

SELECT id, title FROM todos WHERE user_id = $1 ORDER BY created_at DESC;

SELECT t.title, u.email FROM todos t
JOIN users u ON u.id = t.user_id WHERE u.email = $1;

UPDATE todos SET completed = true WHERE id = $1 AND user_id = $2;
DELETE FROM todos WHERE id = $1 AND user_id = $2;
```

SQL injection — parametrelili sorgu

Kullanıcı girdisini ASLA string birleştirmeyle SQL'e gömme. Her zaman \$1,\$2 + değer dizisi. pg değeri 'komut değil, veri' olarak gönderir. Tek doğru savunma.

FAZ 3

Authentication

Kod ilk kez katmanlara ayrılır: *route* → *controller* → *service*. Kullanıcı kaydolur, giriş yapar, token alır.

Klasör mimarisi

```
src/
├── routes/           URL → controller eşleştirme
├── controllers/      req oku, doğrula, service çağır, res yaz
├── services/         hash, token, DB – saf iş mantığı (HTTP'siz)
├── middlewares/      requireAuth – JWT bekçisi
├── schemas/          Zod doğrulama
├── utils/            jwt – token üretme
├── types/            express.d.ts – req.userId tipi
├── db/              pool.ts, schema.sql
└── server.ts         app kur, route bağla, dinle
```

Zod — doğrulama

backend/src/schemas/auth.schema.ts

Zod şemaları + türetilen tipler

```
import { z } from "zod";

export const registerSchema = z.object({
  email: z.string().email(),
  password: z.string().min(8).max(100),
});
export const loginSchema = z.object({
  email: z.string().email(),
  password: z.string().min(1),
});
export type RegisterInput = z.infer<typeof registerSchema>;
```

safeParse

Hata fırlatmaz; {success, data|error} döner.

z.infer<typeof s>

Şemadan otomatik TS tipi. Kural ve tip tek kaynaktan, senkron.

bcrypt — şifre hash'leme

Hash

Tek yönlü, geri çevrilemez. DB sızsa bile şifre korunur.

Salt	Her hash'e rastgele ek. Aynı şifre farklı hash. Rainbow table'ı etkisiz kılar.
Cost factor	Kasıtlı yavaşlık (10-12). Brute-force'u felç eder.
compare	Girilen şifreyi hash ile karşılaştırır (login).

backend/src/services/auth.service.ts*register + login — saf iş mantığı*

```
export async function registerUser(email, password) {
  const passwordHash = await bcrypt.hash(password, 10);
  const result = await pool.query(
    `INSERT INTO users (email, password) VALUES ($1, $2)
    RETURNING id, email, created_at`, [email, passwordHash]);
  return result.rows[0];
}

export async function loginUser(email, password) {
  const result = await pool.query(
    `SELECT id, email, password FROM users WHERE email = $1`, [email]);
  const user = result.rows[0];
  if (!user) return null; // kullanıcı yok
  const ok = await bcrypt.compare(password, user.password);
  if (!ok) return null; // şifre yanlış
  return { id: user.id, email: user.email }; // başarılı
}
```

Güvenlik inceliği

'Kullanıcı yok' ve 'şifre yanlış' ikisi de null döner, aynı mesajla (401) karşılanır. Saldırgan hangi email'lerin kayıtlı olduğunu öğrenemez.

JWT, middleware ve tam auth akışı bir sonraki kısımda (Kısım III) derinlemesine anlatılıyor — en kritik konu olduğu için ayrı bir bölüm ayrıldı.

KISIM III

JWT & Auth Derinlemesine

En çok kafa karıştıran konu, en baştan. Token nedir, neden var, nasıl doğrulanır, ve req.userId'nin gerçek amacı.

BÖLÜM 1

Neden token? Stateless problemi

Bu kısım, projenin en kritik ve en çok kafa karıştıran konusu. Yavaş ve baştan gidiyoruz. Önce neden token'a ihtiyaç duyduğumuzu anlamak gerek — çözüm ondan sonra anlamlı oluyor.

Sorun: HTTP unutkandır

Hatırla: HTTP stateless. Her istek bağımsız; sunucu bir önceki isteği hatırlamaz. Yani kullanıcı login olsa bile, bir sonraki istek geldiğinde sunucu için o yine bir yabancı. Sunucu 'az önce giriş yapan kişi buydu' diye bir şey tutmuyor.

O zaman şu soru doğuyor: 'To-do'larımı getir' diyen birinin kim olduğunu sunucu nasıl bilecek? Herkesin kendi to-do'su var; ben seninkini görememeliyim.

Cevap: kullanıcı her istekte 'ben buyum' diye bir kanıt gösterecek. Bu kanıt JWT. Login'de bir kere alıyor, sonra her korumalı istekte yanında getiriyor.

Lunapark benzetmesi — tüm konuyu tek hikayede

Gişe = login. Bilet parayı ödüyorsun (email+şifre), karşılığında bir bileklik (token) takıyorlar. Bilekliğin taklit edilemez bir mührü var (imza), lunaparkın gizli kalıbıyla basılmış (JWT_SECRET).

İçeride her oyuna binerken kapıdaki görevli (requireAuth middleware) bilekliğine bakar. Gerçekse 'geç'; yoksa/sahteyse 'giremezsin'. Görevli seni tanımıyor — her oyunda yeniden bakar. İşte bu HTTP'nin stateless olması.

Bileklikte 'bu kişi 3 numaralı misafir' yazıyor (payload). Görevli okuyabiliyor ama mühür sayesinde kimse değiştiremiyor.

Lunapark	Bizim sistem
Gişe	POST /auth/login
Bileklik	token (JWT)
Bilekliğin mührü	imza (signature)
Mührün gizli kalıbı	JWT_SECRET
Oyun kapısındaki görevli	requireAuth middleware
'Buyur geç'	next()
'Giremezsin'	401
Bileklikteki 'misafir no'	payload'daki id

BÖLÜM 2

Token'ın anatomisi ve imza

Token üç parçadan oluşur

eyJhbGci...	.	eyJpZCI6Mywi...	.	p8B4dzXR8U9...
header		payload		signature

header	'Bu bir JWT, şu algoritmayla imzalandı' bilgisi.
payload	Asıl içerik: id, email, iat (üretim zamanı), exp (son kullanma).
signature	İmza. Sunucunun gizli anahtarıyla üretilen, sahteciliği önleyen parça.

Payload okunabilir — çözülmüş hali

```
{
  "id": 3,
  "email": "daemon@westeros.com",
  "iat": 1782213959,    // issued at – ne zaman üretildi
  "exp": 1782818759    // expiration – ne zaman geçersiz (7 gün sonra)
}
```

KRİTİK: JWT şifreli değil, İMZALI

Payload (id, email) herkes tarafından okunabilir — jwt.io'ya yapıştır, görürsün. Gizli değil. Peki güvenlik nerede? İmzada. Kimse içeriği değiştiremez. Biri 'id:3'ü 'id:1' yapıp başkasının hesabına girmeye çalışırsa, imza tutmaz çünkü imzayı ancak JWT_SECRET ile üretebilirsin. Sunucu reddeder.

Sonuç: JWT'ye şifre gibi hassas bilgi KONMAZ (okunabilir çünkü). Sadece 'kim olduğu' gibi açık olması sakıncasız bilgiler konur.

Token üretme

backend/src/utls/jwt.ts

signToken — token üretme

```
import jwt from "jsonwebtoken";
const JWT_SECRET = process.env.JWT_SECRET as string;

export function signToken(payload: { id: number; email: string }) {
  return jwt.sign(payload, JWT_SECRET, { expiresIn: "7d" });
}
```

jwt.sign(payload, secret, opts)

Payload'ı secret ile imzalayıp token üretir.

expiresIn: '7d'

Token 7 gün geçerli. Süre dolunca geçersiz — çalınsa bile sonsuza kadar işe yaramaz.

JWT_SECRET

İmza anahtarı. .env'de. Sızarsa biri sahte token üretebilir — DB şifresi kadar kritik.

BÖLÜM 3

requireAuth — satır satır

Middleware, isteğin controller'a varmadan uğradığı ara durak (bekçi). requireAuth token'ı kontrol eder: geçerliyse içeri alır ve kullanıcının kim olduğunu isteğe ilştirir; değilse 401 ile çevirir.

backend/src/middlewares/requireAuth.ts

Korumalı route'ların bekçisi

```
import { Request, Response, NextFunction } from "express";
import jwt from "jsonwebtoken";
const JWT_SECRET = process.env.JWT_SECRET as string;

export function requireAuth(req: Request, res: Response, next: NextFunction)
{
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ error: "No token provided" });
  }

  const token = authHeader.split(" ")[1];

  try {
    const payload = jwt.verify(token, JWT_SECRET) as { id: number; email:
string };
    req.userId = payload.id;
    next();
  } catch {
    return res.status(401).json({ error: "Invalid or expired token" });
  }
}
```

Her satır ne yapıyor

- 1 — header'ı al** req.headers.authorization → 'Bearer ey]...' metnini al.
- 2 — kontrol** Header var mı + 'Bearer ' ile başlıyor mu? Biri eksikse 401, dur. next() çağrılmaz → controller'a gitmez.
- 3 — token'ı ayıkla** split(' ')[1] → metni boşluktan böl, ikinci parça (saf token). 'Bearer ' öneki gider.
- 4 — DOĞRULA** jwt.verify(token, secret): üç şeyi birden kontrol eder → (a) imza gerçek mi, (b) içerik değiştirilmiş mi, (c) süresi dolmuş mu. Geçerliyse payload döner; değilse HATA fırlatır → catch'e atlar.

5 — req'e iliştiir req.userId = payload.id. Token'dan çıkan 'kim' bilgisini req'in üstüne yapıştırır.

6 — next() Geçiş izni. İstek controller'a ilerler.

next() nedir

Bir fonksiyon: 'işim bitti, sıradakine geç' demek. Çağrılırsa istek controller'a ilerler; çağrılmazsa istek burada takılır. Middleware'i controller'dan ayıran budur — controller yolun sonu, middleware ortadaki durak. Token geçerliyse next() çağrılır; geçersizse (header yok ya da catch) 401 + return, next() YOK.

req.userId = payload.id — neden bu satır?

Bu satır en çok kafa karıştıran yer. Anlamı, bir sonraki adımda ne olacağına bakınca açılıyor.

requireAuth bittikten sonra istek controller'a gidiyor (asıl işi yapan yer — örn. 'to-do'larımı getir' fonksiyonu). O controller'ın bir şeyi bilmesi gerek: bu isteği yapan kullanıcı kim? Çünkü 'to-do'ları getir' derken, kimin to-do'larını getireceğini bilmeli.

O 'kim' bilgisi şu an payload.id'nin içinde (yani 3). Token'dan çıkardık. Ama bu bilgi sadece requireAuth'un içinde, yerel bir değişkende. Controller bunu göremez. İşte req.userId = payload.id tam bunu çözüyor: id'yi isteğin (req) üstüne yapıştırıyoruz.

Neden req'e? Çünkü req nesnesi requireAuth'tan controller'a birlikte gidiyor. Aynı istek, aynı req, kapıdan geçip içeri giriyor. Üstüne not yapıştırırsan, içerideki controller o notu okuyabilir.

Lunapark görevlisi damgası

Görevli bilekliğine baktı, gerçek olduğunu gördü, içinde '3 numaralı misafir' yazdığını okudu. Seni içeri alırken eline görünmez bir damga basıyor: '3'. İçerideki oyun operatörü (controller) bu damgaya bakıp 'aa, 3 numara, ona ait şeyleri getireyim' diyor. req.userId = payload.id = o damgayı basma işlemi.

Faz 4'te okuma ucu (önizleme)

Yazma ucunu gördün (requireAuth ilişkiliyor). Okuma ucu Faz 4'te gelecek. To-do controller'ında şöyle olacak:

```
// todos.controller.ts (Faz 4)
const userId = req.userId; // ← bekçinin bastığı damgayı OKU
// pool.query("SELECT * FROM todos WHERE user_id = $1", [userId])
// yani: SADECE bu kullanıcının to-do'larını getir
```

İşte o an — yazma (requireAuth) ve okuma (controller) uçları birlikte görününce — req.userId = payload.id satırı tam oturacak. Tek başına soyut; zincirin tamamını görünce somut.

req.userId için tip eklemesi

req.userId Express'in standart alanı değil; TS itiraz eder ('Property userId does not exist'). Bir declaration dosyasıyla Request tipini genişletiriz (type augmentation):

backend/src/types/express.d.ts

Request tipine userId ekleme

```
declare global {
  namespace Express {
    interface Request {
      userId?: number; // ? = opsiyonel (sadece korumalı route'larda dolu)
    }
  }
}
export {};
```

Tam akış — korumalı istek baştan sona

```
İSTEK: GET /todos + Authorization: Bearer eyJ...
```

- ▼ server.ts body parse + /todos → router
- ▼ todos.routes.ts router.get("/", requireAuth, getTodos)
 → önce bekçi, sonra controller
- ▼ requireAuth.ts header al → Bearer kontrol → token ayıkla
 jwt.verify(token, JWT_SECRET)
 └─ geçersiz → 401, DUR
 └─ geçerli → req.userId = payload.id → next()
- ▼ todos.controller.ts
 const userId = req.userId ← damgayı oku
 service'i userId ile çağır
- ▼ todos.service.ts
 pool.query("... WHERE user_id = \$1", [userId])
- ▼ pool.ts → Neon bu kullanıcının satırlarını getir
- ▲ sonuç ters tırmanır → res.json(todos) → İSTEK SAHİBİNE

import 'dotenv/config' tuzağı

server.ts'in EN ÜSTÜNDE olmalı. Bu satır .env'i her şeyden önce yükler; jwt ve requireAuth process.env'i dolu bulur. Üstte olmazsa JWT_SECRET boş gelir, token doğrulama 'Invalid token' verir — yaşadığın hata buydu.

BÖLÜM 4

Sık sorulan sorular ve tuzaklar

Token nereden geliyor?	Login'de sunucu üretip sana verir. Sen saklarsın (gerçek uygulamada tarayıcı; Postman'de elle).
Neden her istekte gönderiyorum?	Sunucu seni hatırlamıyor (stateless). Her seferinde 'ben buyum' kanıtı gerek.
Şifre gibi mi?	Hayır. Şifreyi sadece login'de bir kere verirsin. Sonra şifre değil, token gönderirsin. Token = 'şifremi doğru girdim, işte kanıtı'.
Token'ı değiştirebilir miyim?	Payload'ı evet (okunabilir), ama imza tutmaz → sunucu reddeder. İmzayı JWT_SECRET olmadan üretemezsin.
'Invalid token' alıyorum?	En sık sebep: token üretilirken kullanılan secret ile doğrularken kullanılan secret farklı. Genelde dotenv geç yüklenmesinden. <code>import 'dotenv/config'</code> en üstte mi?
401 vs 403?	401: kimliğini kanıtlayamadın (token yok/geçersiz). 403: kimsin biliyorum ama bu işe yetkin yok.
Postman'de nereye?	Token Headers sekmesine. Key: Authorization, Value: 'Bearer ' + token. Params'a DEĞİL.

Hatırlanacak üç cümle

Özet

1. Login = bileklik (token) alma, bir kere.
2. `requireAuth` = bilekliği kapıda gösterme, her korumalı istekte.
3. HTTP unutkan (stateless) olduğu için her seferinde kimliğini yeniden kanıtlarsın — token bunu mümkün kılar.

KISIM IV

Referanslar

Hızlı başvuru: dosya haritası, istek döngüsü, komutlar, kavram sözlüğü.

BÖLÜM 1

Dosya haritası

Şu ana kadar yarattığın her dosya ve görevi:

Dosya	Görevi
package.json	Proje kimliği, bağımlılıklar, script'ler
tsconfig.json	TypeScript derleyici ayarları
.gitignore	Git'in yok sayacağı dosyalar (node_modules, .env)
.env	Gerçek sırlar (DATABASE_URL, JWT_SECRET) — git'te YOK
.env.example	Sır şablonu (değersiz) — git'te VAR
src/server.ts	Giriş: app kur, middleware + route bağla, dinle
src/db/pool.ts	PostgreSQL connection pool
src/db/schema.sql	Tablo tanımları — yapının kaynak gerçeği
src/schemas/auth.schema.ts	Zod doğrulama + türetilen tipler
src/services/auth.service.ts	register/login iş mantığı — HTTP'siz
src/controllers/auth.controller.ts	req oku → doğrula → service → res yaz
src/routes/auth.routes.ts	POST /register, /login eşleştirmeleri
src/middlewares/requireAuth.ts	JWT bekçisi — token kontrol, req.userId iliştiir
src/utlis/jwt.ts	signToken — JWT üretme
src/types/express.d.ts	Request tipine userId ekleme

BÖLÜM 2

Komut ve kavram sözlüğü

Sık komutlar

```
# npm
npm install <paket>      dependency ekle
npm install -D <paket>   devDependency ekle
npm run dev               geliştirme (tsx watch)
npm run build             production derleme (tsc)
npm start                 derlenmiş kodu çalıştır

# git
git status                durumu gör
git add .                 stage'le
git commit -m "...       commit'le
git log --oneline         commit geçmişi
```

HTTP status code'ları (bu projede)

Kod	Anlam	Nerede
200	OK	Başarılı login, GET
201	Created	register sonrası yeni kullanıcı
400	Bad Request	Zod doğrulama hatası
401	Unauthorized	Token yok/geçersiz, login başarısız
404	Not Found	Kaynak yok (Faz 4'te to-do)
500	Internal Server Error	Sunucu hatası

Anahtar kavramlar — tek cümlede

REST	HTTP üzerine kurulu, kaynak-odaklı API tasarım üslubu.
Stateless	Sunucu önceki isteği hatırlamaz; her istek kendi kanıtını taşır.
Middleware	İstek controller'a varmadan geçtiği ara fonksiyon.
Hash	Tek yönlü, geri çevrilemez dönüşüm. Şifre saklamak için.
Salt	Her hash'e eklenen rastgele değer; rainbow table'ı etkisiz kılar.
JWT	Sunucunun imzaladığı kimlik token'ı. header.payload.signature.
Connection pool	Yeniden kullanılan DB bağlantıları havuzu.

Foreign key	Bir tablonun başka tablonun PK'sını işaret etmesi.
Parametrelili sorgu	\$1,\$2 + değer dizisi. SQL injection'a karşı savunma.
type erasure	TS tipleri runtime'da silinir; kontrol yalnız compile-time.
Idempotency	Aynı işlemi tekrar yapmak tek yapmakla aynı sonucu verir.
Referential integrity	FK'nin sağladığı, parent-child tutarlılığı garantisi.

Commit geçmişi (Faz 1-3)

```
feat(auth): add register, login, JWT auth, and requireAuth middleware
feat(db): add postgres schema, connection pool, and db-health endpoint
feat(backend): scaffold express + typescript server
```

Bu rehber Faz 1-3'ü kapsar. Sıradaki durak Faz 4 — To-Do CRUD. Orada req.userId'yi ilk kez okuyup 'herkes yalnız kendi to-do'sunu görür' mantığını canlı kuracağız; JWT/auth kavramı tam yerine oturacak.

İyi çalışmalar, Blackfyre33.

KISIM V

Veri Nereye Gidiyor?

Her parça ne alır, ne döndürür, ve dönen şey sonra nerede kullanılır. Bir de token'ı açıp içine bakıyoruz.

BÖLÜM 1

Dönen değerler zinciri

En çok kafa karıştıran şey: 'bir şey oluşturuyoruz ama sonucu ne dönüyor, o dönen şey nereye gidiyor?' Bu bölüm her parçanın GİRDİ → ÇIKTI'sını ve çıktının bir sonraki durağını net gösteriyor.

Login akışı — girdi ve çıktılar

```

KULLANICI GÖNDERİR
{ email, password }
|
▼ loginSchema.safeParse(req.body)
|   GİRDİ: ham req.body
|   ÇIKTI: { success: true, data: {email, password} }
|           (geçersizse { success: false }) → 400
|
▼ loginUser(email, password)
|   GİRDİ: email, password (saf veri)
|   içeride: DB'den kullanıcıyı bul + bcrypt.compare
|   ÇIKTI: { id, email } (password ATILIR, id EKLENİR)
|           (kullanıcı yok / şifre yanlış → null) → 401
|
▼ signToken(user)
|   GİRDİ: { id, email }
|   ÇIKTI: "eyJhbGci..." (imzalı token dizesi)
|
▼ res.json({ token })
|   ÇIKTI: kullanıcıya giden HTTP cevabı
|
▼ KULLANICI ALIR
{ token: "eyJhbGci..." }

```

Dikkat: her okta veri DEĞİŞİYOR

Başta {email, password} vardı. loginUser'dan sonra {id, email} oldu (password gitti, id geldi). signToken'dan sonra bir dize (token) oldu. Her fonksiyon bir şey alıp, başka bir şey döndürüyor — ve o dönen şey, bir sonrakinin girdisi oluyor. Zincir böyle akıyor.

Her fonksiyon ne döndürür — tablo

Fonksiyon	Ne ALIR	Ne DÖNDÜRÜR
safeParse(body)	ham veri	{success, data} ya da {success:false}
bcrypt.hash(pw, 10)	düz şifre	hash dizesi (\$2b\$10\$...)
bcrypt.compare(pw, hash)	şifre + hash	true / false
registerUser(email, pw)	email + şifre	{ id, email, created_at }

<code>loginUser(email, pw)</code>	email + şifre	{ id, email } ya da null
<code>signToken({id, email})</code>	kullanıcı bilgisi	token dizesi
<code>jwt.verify(token, secret)</code>	token + secret	payload {id, email, iat, exp}
<code>pool.query(sql, params)</code>	SQL + değerler	{ rows: [...], rowCount }

pool.query özel — .rows'a dikkat

`pool.query` bir nesne döndürür: { rows, rowCount, ... }. Asıl veriler `.rows` dizisindedir. Tek satır beklerken `result.rows[0]` kullanırsın. Boşsa `rows[0]` undefined olur — `loginUser`'da 'kullanıcı yok' kontrolü tam buydu.

requireAuth akışı — girdi ve çıktılar

```

İSTEK GELİR
  header: "Authorization: Bearer eyJhbGci..."
  |
  ▼ req.headers.authorization
    ÇIKTI: "Bearer eyJhbGci..." (ya da undefined)
  |
  ▼ authHeader.split(" ")[1]
    GİRDİ: "Bearer eyJhbGci..."
    ÇIKTI: "eyJhbGci..." (saf token)
  |
  ▼ jwt.verify(token, JWT_SECRET)
    GİRDİ: token + secret
    ÇIKTI: { id: 3, email, iat, exp } (payload)
           (sahte/süresi dolmuş → HATA fırlatır → 401)
  |
  ▼ req.userId = payload.id
    payload.id (3) → req'in üstüne yazılır
    ÇIKTI yok — req nesnesi DEĞİŞTİRİLİR
  |
  ▼ next()
    ÇIKTI yok — sadece 'geç' sinyali
  |
  ▼ CONTROLLER (req.userId artık dolu: 3)

```

İki tür 'çıktı' var

Bazı satırlar değer döndürür (jwt.verify → payload). Bazıları değer döndürmez, bir yan etki yapar: req.userId = ... req'i değiştirir, next() akışı ilerletir, res.json() cevabı gönderir. 'Sonuç ne döndü?' sorusunun cevabı her zaman bir değer değil — bazen bir eylemdir.

BÖLÜM 2

Token'ı açıp içine bakmak

Ürettiğimiz token karmaşık görünür ama aslında noktalarla ayrılmış üç parçadır.
Her parçayı açalım.

```

eyJhbGci...VCJ9 . eyJpZCI6Mywi...NTl9 . V9NUNxTDZT8l...kAM
header           payload           signature

```

Parça 1 — Header

Çözülünce (Base64, herkes çözebilir):

```
{
  "alg": "HS256",    // imza algoritması
  "typ": "JWT"       // bu bir JWT
}
```

Token'ın kimlik bilgisi: ne olduğu ve nasıl imzalandığı. `jwt.verify` buraya bakıp doğru algoritmayla kontrol eder.

Parça 2 — Payload (asıl içerik)

```
{
  "id": 3,                // ← BİZİM koyduğumuz
  "email": "daemon@...", // ← BİZİM koyduğumuz
  "iat": 1782213959,      // ← jwt ekledi: üretim zamanı
  "exp": 1782818759       // ← jwt ekledi: son kullanma (iat + 7 gün)
}
```

id, email signInToken'a verdiğimiz bilgi. Kullanıcının kim olduğu.

iat 'issued at' — token ne zaman üretildi (Unix zaman damgası).

exp 'expiration' — ne zaman geçersiz. expiresIn: '7d' bunu belirler. jwt.verify süreyi buradan kontrol eder.

Payload okunabilir — şifreli değil

Bu üç-dört alan herkes tarafından çözülebilir (jwt.io'ya yapıştır, görsün). Gizli değil! O yüzden şifre gibi hassas bilgi **KONMAZ**. Güvenlik içeriği gizlemekte değil, değiştirilemez kılmakta — bir sonraki parçada.

Parça 3 — Signature (imza)

Bu parça okunabilir bilgi değil, bir mühür. Şöyle üretilir:

```
signature = HS256( header + "." + payload, JWT_SECRET )
```

Header ve payload'ı al, JWT_SECRET ile özel bir hesaba sok, çıkan sonuç imza. İmzayı üretmek için secret şart — secret'ı olmayan geçerli imza üretemez.

İmzanın koruması — somut senaryo

Saldırgan payload'daki id: 3'ü id: 1 yapıp başkasının hesabına girmek istedi. Payload'ı değiştirebilir (açık). AMA payload değişince ona uygun yeni imza üretmesi lazım — bunun için JWT_SECRET gerekiyor, onda yok. jwt.verify imzayı kontrol edince 'içerik imzayla uyuşmuyor' der ve reddeder. İşte token'ın güvenliği bu: içerik açık ama sahtecilik imkansız.

Özet — üç parça

Parça	İçerik	Kim koydu
header	alg, typ	jwt otomatik
payload	id, email, iat, exp	id+email biz; iat+exp jwt
signature	mühür (HS256 + secret)	jwt, secret ile